

1. ¿Cómo se comporta el Hardware de forma automática?

El procesador principal no va a cambiar los canales del multiplexor con código lento (`digitalWrite`). En su lugar, el co-procesador **PIO** se encarga de manejar el multiplexor en microsegundos, introduce un retraso de seguridad para estabilizar la señal, le ordena al **ADC** medir y el **DMA** guarda el resultado directamente en la memoria RAM.

2. Instrucciones breves de Programación (En C Puro)

- **Paso A: Programar el bloque PIO (El Orquestador)**
 - Crea un script de PIO corto en ensamblador (`.pio`) que use una instrucción `OUT pins, 4` apuntando a los pines de selección S0-S3 del multiplexor.
 - Configura el programa para que, inmediatamente después de cambiar el canal, use un ciclo de retraso por hardware de exactamente **15 microsegundos**. Esto le da tiempo a la capacitancia parásita del MUX de estabilizarse.
 - Programa la última instrucción del bloque para que envíe un pulso interno de inicio al ADC (*Hardware Trigger*).
- **Paso B: Configurar el ADC (El Digitalizador)**
 - Inicializa el ADC y selecciona el pin analógico donde está soldada la salida `SIG` del multiplexor.
 - Habilita la FIFO interna del ADC (`adc_fifo_setup`) y actívala para que trabaje con solicitudes de DMA (`DREQ`). Cada vez que una muestra analógica esté lista (tarda $2\ \mu\text{s}$), se enviará automáticamente a la FIFO.
- **Paso C: Configurar el DMA (El Transportador)**
 - Inicializa un canal del DMA configurando su dirección de lectura (`read_addr`) apuntando directamente al registro físico de la FIFO del ADC (`&adc_hw->fifo`).
 - Configura la dirección de escritura (`write_addr`) apuntando al arreglo de memoria de tu código: `uint16_t hall_samples[14]`.
 - Establece el tamaño de transferencia en 14 (`transfer_count = 14`) y activa el incremento automático de dirección de destino para que llene el arreglo en orden.
 - Configura el disparador del DMA amarrado al evento `DREQ_ADC`.
- **Paso D: El Lazo de Control (La Bandera)**
 - Habilita la interrupción por hardware del DMA para cuando complete las 14 transferencias (`dma_channel_set_irq0_enabled`).
 - En la rutina de la interrupción (ISR), lo único que debes programar es levantar una bandera de software: `FLAG_ADC_COMPLETE = 1`.
 - Tu código principal del Core 0 solo hará un `polling` esperando a que esa bandera sea 1 para proceder a leer la IMU y entregarle los datos limpios al Core 1.